

P3369R0

constexpr for uninitialized_default_construct

Published Proposal, 2024-07-28

Author:

[Giuseppe D'Angelo](#)

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

This paper proposes to add `constexpr` to `uninitialized_default_construct`.

Table of Contents

- 1 Changelog
- 2 Motivation and scope
- 3 Impact on the Standard
- 4 Proposed Wording
 - 4.1 Feature-testing macro
 - 4.2 Wording
- 5 Acknowledgements

References

§ 1. Changelog

- R0
 - First submission

§ 2. Motivation and scope

[P2283R2] ("constexpr for specialized memory algorithms") proposes to add `constexpr` to most specialized memory algorithms:

- `uninitialized_value_construct`;
- `uninitialized_copy`;
- `uninitialized_move`;
- `uninitialized_fill`.

All the overloads of these algorithms in namespaces `std` and `std::ranges`, as well as their `_n` variants, are affected by [P2283R2]; the only exceptions are the parallel overloads (cf. [P2902R0], "`constexpr` Parallel Algorithms").

One algorithm family is missing: `uninitialized_default_construct`. The reason for it is that it was impossible to perform default construction in a `constexpr` context; `std::construct_at` always performs *value* initialization, and indeed, all the proposed algorithms are specified in terms of calls to `std::construct_at`.

With the adoption of [P2747R2] ("`constexpr` placement new") it is now possible to perform such default construction during constant evaluation, without the need of changing how `uninitialized_default_construct` is specified. Therefore, this paper simply proposes to add `constexpr` to those algorithms.

§ 3. Impact on the Standard

This proposal is a pure library addition. The necessary changes to core language have already been adopted.

§ 4. Proposed Wording

All the proposed changes are relative to [\[N4986\]](#).

§ 4.1. Feature-testing macro

In [version.syn], modify the value of the `__cpp_lib_raw_memory_algorithms` to match the date of adoption of the present proposal.

§ 4.2. Wording

Modify [memory.syn] as follows:

```
template<class NoThrowForwardIterator>
    constexpr
    void uninitialized_default_construct(NoThrowForwardIterator first,
                                         NoThrowForwardIterator last);
template<class ExecutionPolicy, class NoThrowForwardIterator>
    void uninitialized_default_construct(ExecutionPolicy&& exec,
                                         NoThrowForwardIterator first,
                                         NoThrowForwardIterator last);
template<class NoThrowForwardIterator, class Size>
    constexpr
    NoThrowForwardIterator
        uninitialized_default_construct_n(NoThrowForwardIterator first, Size n);
template<class ExecutionPolicy, class NoThrowForwardIterator, class Size>
    NoThrowForwardIterator
        uninitialized_default_construct_n(ExecutionPolicy&& exec,
                                         NoThrowForwardIterator first, Size n);

namespace ranges {
    template<nothrow-forward-iterator I, nothrow-sentinel-for<I> S>
        requires default_initializable<iter_value_t<I>>
            constexpr
            I uninitialized_default_construct(I first, S last);
    template<nothrow-forward-range R>
        requires default_initializable<range_value_t<R>>
            constexpr
            borrowed_iterator_t<R> uninitialized_default_construct(R&& r);
```

```

template<nothrow-forward-iterator I>
    requires default_initializable<iter_value_t<I>>
    constexpr
        I uninitialized_default_construct_n(I first, iter_difference_t<I> n);
}

```

Modify [uninitialized.construct.default] as follows:

```

template<class NoThrowForwardIterator>
    constexpr
        void uninitialized_default_construct(NoThrowForwardIterator first, NoThrowForwardIterator last);

```

```

namespace ranges {
    template<nothrow-forward-iterator I, nothrow-sentinel-for<I> S>
        requires default_initializable<iter_value_t<I>>
        constexpr
            I uninitialized_default_construct(I first, S last);
    template<nothrow-forward-range R>
        requires default_initializable<range_value_t<R>>
        constexpr
            borrowed_iterator_t<R> uninitialized_default_construct(R&& r);
}

```

```

template<class NoThrowForwardIterator, class Size>
    constexpr
        NoThrowForwardIterator uninitialized_default_construct_n(NoThrowForwardIterator first, Size n);

```

```

namespace ranges {
    template<nothrow-forward-iterator I>
        requires default_initializable<iter_value_t<I>>
        constexpr
            I uninitialized_default_construct_n(I first, iter_difference_t<I> n);
}

```

§ 5. Acknowledgements

Thanks to Michael Schellenberger Costa for [P2283R2] and Barry Revzin for [P2747R2].

Thanks to KDAB for supporting this work.

All remaining errors are ours and ours only.

§ References

§ Informative References

[N4986]

Thomas Köppe. *Working Draft, Programming Languages — C++*. 16 July 2024. URL: <https://wg21.link/n4986>

[P2283R2]

Michael Schellenberger Costa. *constexpr for specialized memory algorithms*. 26 November 2021. URL: <https://wg21.link/p2283r2>

[P2747R2]

Barry Revzin. *constexpr placement new*. 19 March 2024. URL: <https://wg21.link/p2747r2>

[P2902R0]

Oliver Rosten. *constexpr 'Parallel' Algorithms*. 17 June 2023. URL: <https://wg21.link/p2902r0>